

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: UI-тестирование

Студенты гр. 4388

Арефьев И. Д.
Денисенко В. С.
Донцова И. Е.
Шарапов Д. Р.

Руководитель

Шевелева А. М.

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Денисенко В. С.

Группа: 4388

Тема практики: UI-тестирование

Задание на практику:

Задание на практику заключалось в разработке **автоматизированных тестов** для проверки пользовательского интерфейса (UI-тестирования) веб-сайта Rutube. Основная работа включала изучение основ автоматизации тестирования на языке Java с использованием инструментов Selenide, а также проектирование архитектуры проекта на основе Page Object Model.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент	_____	Денисенко В. С.
Руководитель	_____	Шевелева А. М.

АННОТАЦИЯ

В данном отчете представлена работа по созданию автоматического UI-тестирования Rutube. Проект разработан на языке Java с использованием библиотек Selenide, JUnit 5 и системы сборки Maven.

Проект устроен по принципу Page Object Model, то есть весь код для удобства разделен по папкам: базовые настройки (base), элементы интерфейса (elements), описание страниц сайта (pages) и сами автотесты (tests). Такое разделение позволяет отделить логику взаимодействия с сайтом от логики самих проверок, что сильно упрощает обновление кода при изменении дизайна Rutube. Также в проекте реализована автоматическая авторизация с помощью чтения сессионных кук из JSON-файла.

В практической части отчета описана реализация 10 UI-тестов, которые проверяют ключевые функции Rutube. Тесты покрывают поиск видеороликов и каналов, применение фильтров, оформление подписок, добавление видео в плейлисты «Смотреть позже» и «Понравилось», отправку жалоб, копирование ссылок, а также управление качеством и полноэкранным режимом в самом видеоплеере. Все разработанные тесты успешно работают.

СОДЕРЖАНИЕ

АННОТАЦИЯ	3
ВВЕДЕНИЕ	5
1. ПОСТАНОВКА ЗАДАЧИ	7
1.1 Предметная область.....	7
1.2 Задачи практики.....	7
2. РЕЗУЛЬТАТЫ РАБОТЫ	8
2.1. Описание реализуемых тестов	8
2.1.1 Проверка поиска	8
2.1.2 Проверка работы с каналами.....	11
2.1.3 Проверка взаимодействия с видео	11
2.1.4 Результат выполнения всех тестов	17
2.2 Описание классов и методов.....	18
2.2.1. Блок базовых классов (base).....	18
2.2.2 Блок элементов интерфейса (elements)	19
2.2.3 Блок страниц (pages).....	21
2.2.4 Блок тестов (tests)	24
2.2.5 Блок утилит (utils).....	26
2.2.6 Диаграмма классов (UML)	26
2.3 Индивидуальная работа	29
2.3.1 Составление чек-листа для проверок	29
2.3.2 Настройка автоматической авторизации	29
2.3.3 Рефакторинг по требованиям преподавателя.....	29
2.3.4 Исправление и стабилизация тестов.....	30
3. ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	31
4. ОТЗЫВ РУКОВОДИТЕЛЯ	33
ЗАКЛЮЧЕНИЕ	34
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36

ВВЕДЕНИЕ

Предметной областью данной практики является автоматизация тестирования для обеспечения качественной работы веб-приложений, таких как Rutube. Автоматизированное UI-тестирование позволяет имитировать действия реальных пользователей, находить ошибки при обновлении дизайна или логики сайта, а также значительно ускоряет процесс проверки работоспособности.

Целью учебной практики является создание 10 автоматизированных UI-тестов для проверки ключевых функций и интерфейса Rutube с использованием Java, Selenide, JUnit 5.

Задачи учебной практики:

1. Изучение автоматизации с использованием языка Java и библиотеки Selenide.
2. Проектирование структуры проекта по принципу Page Object Model для разделения логики проверок и элементов интерфейса
3. Программное описание основных страниц и компонентов сайта
4. Настройка автоматической авторизации через JSON-файлы с куками
5. Написание 10 автотестов для проверки различных сценариев использования платформы.

Поставленное задание заключалось в командной разработке автотестов для проверки интерфейса Rutube. Нам необходимо было спроектировать удобную структуру папок, подключить нужные библиотеки, написать 10 автотестов для ключевых функций и настроить автоматический вход в аккаунт. Также важной частью задания была совместная отладка кода и исправление всех ошибок по замечаниям преподавателя.

Вклад каждого участника команды:

- Арефьев И. Д. (тимлид): создал общий репозиторий на GitHub, написал поясняющие комментарии в коде для разных функций, а также исправил ошибки в нескольких тестах, чтобы они запускались без сбоев.
- Денисенко В. С.: составила чек-лист для проверок, настроила автоматическую авторизацию через чтение сессионных кук из JSON-файла, исправила несколько тестов, чтобы они корректно работали, и внесла изменения по требованиям преподавателя.
- Донцова И. Е.: написала основной каркас для всего проекта, создала структуру папок по шаблону Page Object Model и настроила файл сборки проекта pom.xml, а также внесла изменения в код по замечаниям преподавателя.
- Шарапов Д. Р.: занимался доработкой и чисткой кода, исправил логику работы видеоплеера и поисковых фильтров, добавил валидацию запросов, а также внес исправления по замечаниям преподавателя.

1. ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Предметной областью данной практики является автоматизация тестирования пользовательского интерфейса для обеспечения качественной работы веб-приложений, таких как Rutube. Автоматизированное UI-тестирование позволяет имитировать действия реальных пользователей. Программа сама открывает браузер, вводит текст, кликает по элементам и проверяет, правильно ли отработал сайт. Автотесты помогают быстрее находить ошибки после изменения дизайна или логики сайта, а также значительно ускоряет процесс проверки работоспособности.

1.2 Задачи практики

В рамках прохождения учебной практики были поставлены следующие задачи:

1. Изучение автоматизации с использованием языка Java и библиотеки Selenide.
2. Проектирование структуры проекта по принципу Page Object Model для разделения логики проверок и элементов интерфейса
3. Программное описание основных страниц и компонентов сайта
4. Настройка автоматической авторизации через JSON-файлы с куками
5. Написание 10 автотестов для проверки различных сценариев использования платформы.

2. РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Описание реализуемых тестов

В этом разделе описывается работа 10 написанных автотестов для Rutube. Для удобства все тесты поделены на три группы: поиск, работа с каналами и проверка самого видеоплеера.

2.1.1 Проверка поиска

В этой группе тестов проверяется, как сайт ищет видео. Здесь мы смотрим, правильно ли обновляются результаты, если поменять слово в строке поиска, не ломается ли страница от пустого запроса, и корректно ли работают фильтры по дате добавления и длительности ролика.

Тест 2. Применение фильтров: Тест вводит запрос «новости», нажимает на кнопку «Найти» (рис. 1), открывает панель фильтров и выбирает значения «За сегодня» и «20–60 минут» (рис. 2). Затем он берет первое видео из списка и проверяет, что оно действительно опубликовано сегодня и его длительность попадает в этот диапазон.



Рисунок 1 – Ввод поискового запроса «Новости»

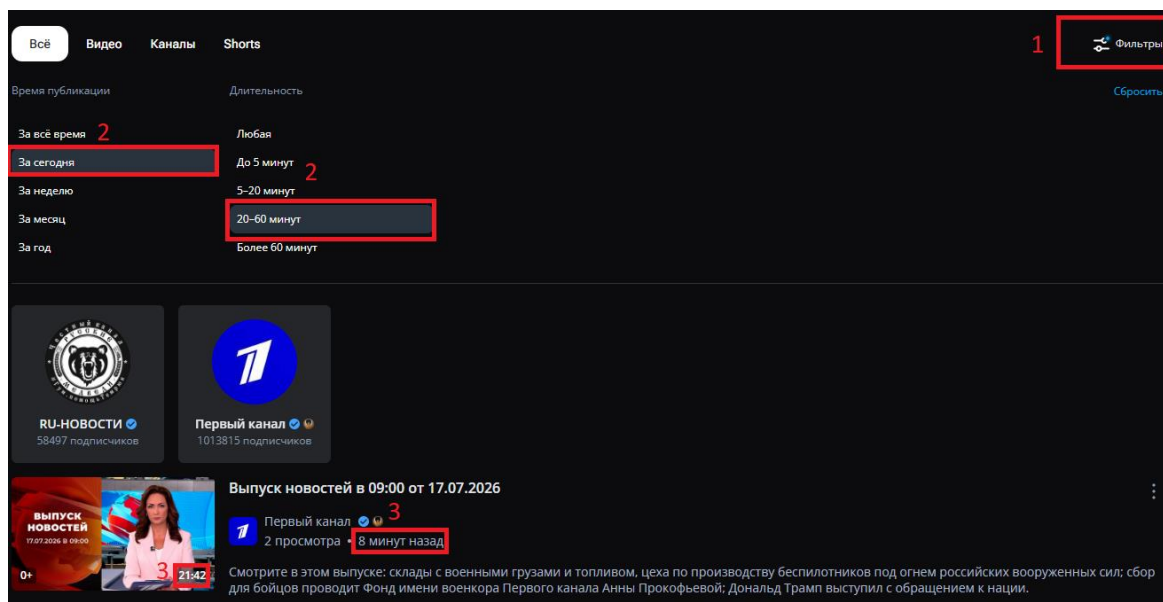


Рисунок 2 – Применение фильтров и проверка видео на соответствие

Тест 3. Изменение запроса: Тест ищет видео по слову «музыка» и запоминает название первого ролика (рис. 3). После этого он очищает поле ввода с помощью крестика, вводит запрос «кино» и проверяет, что поле поиска обновилось, а результаты на странице изменились (рис. 4).

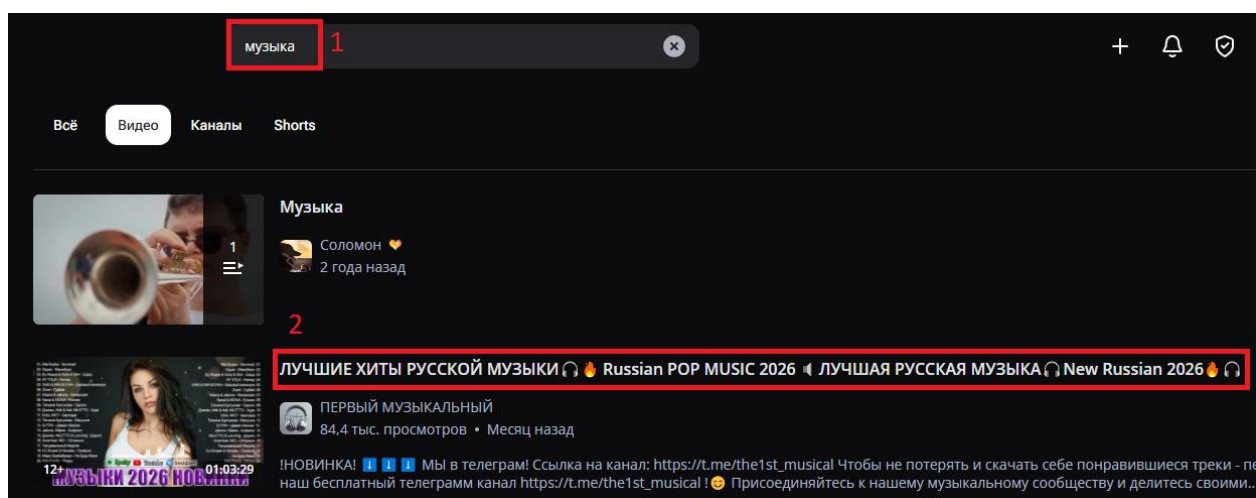


Рисунок 3 – Ввод поискового запроса «Музыка» и запоминание названия видео

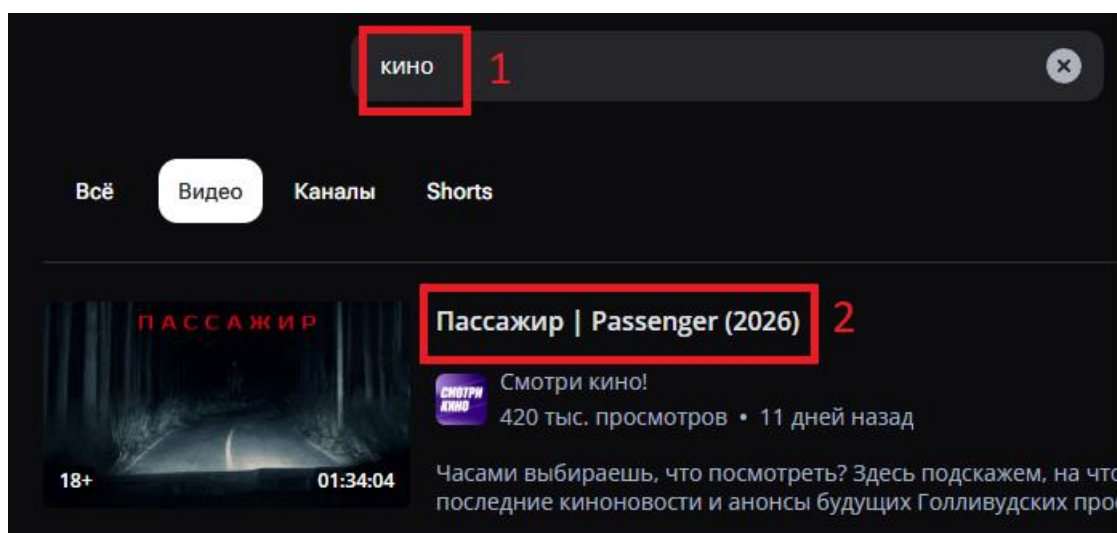


Рисунок 4 – Ввод поискового запроса «Кино» и проверка названия видео

Тест 5. Поиск с пустым запросом: Проверяет реакцию сайта на пустую строку поиска. Тест отправляет пустой запрос, через левое меню переходит в раздел «В топе» (рис. 5), а затем кликает на логотип, чтобы вернуться на главную страницу (рис.6). Проверяется, что интерфейс не сломался и главная страница открылась.

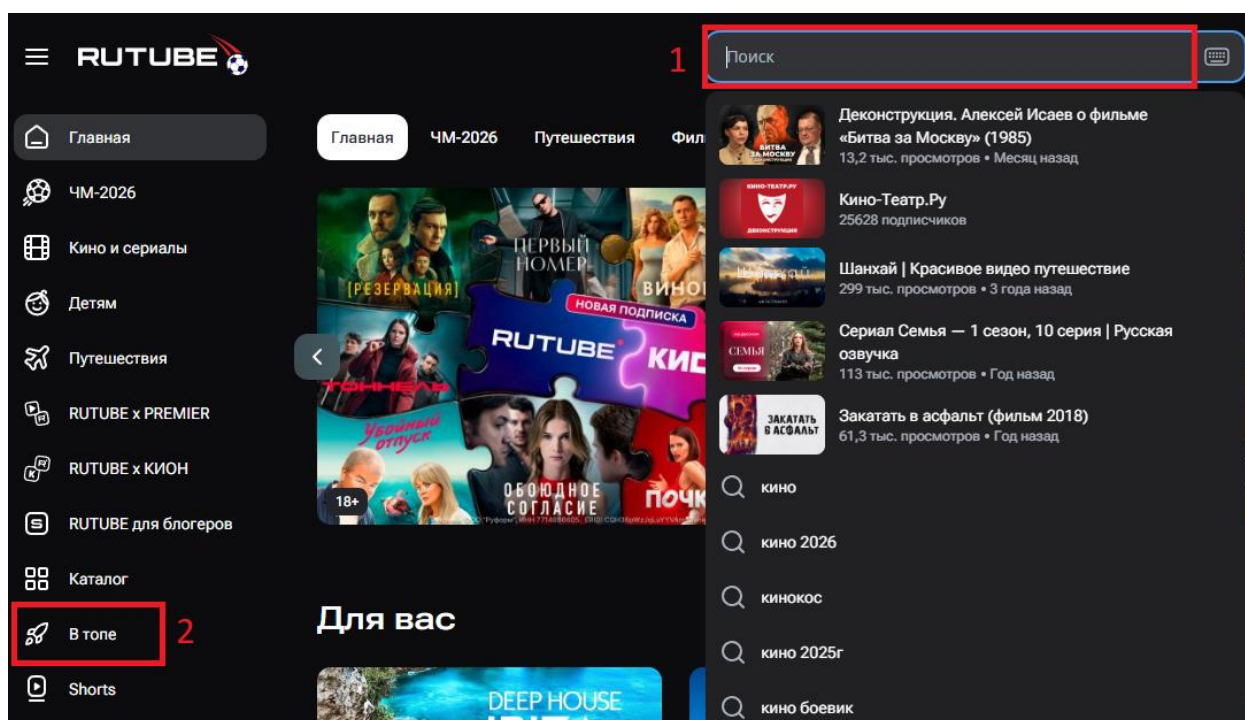


Рисунок 5 – Ввод пустого поискового запроса и переход в раздел «В топе»

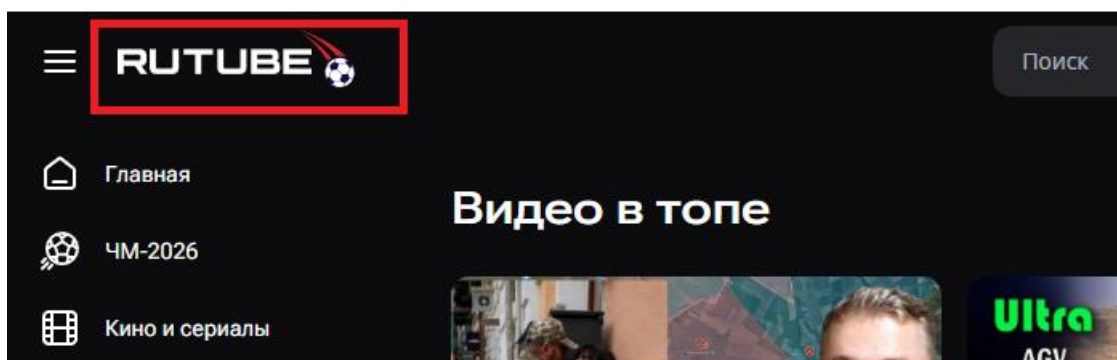


Рисунок 6 – Переход на главную страницу из раздела «В топе»

2.1.2 Проверка работы с каналами

Здесь тест, который проверяет, что можно найти нужный канал, подписаться на него и проверить статус подписки.

Тест 6. Поиск канала и оформление подписки: Тест вводит запрос «Матч ТВ», переключается на вкладку «Каналы» и нажимает кнопку «Подписаться» у первого найденного канала (рис. 7). Затем он переходит в профиль этого канала и проверяет, что кнопка поменяла статус на «Вы подписаны» (рис. 8).

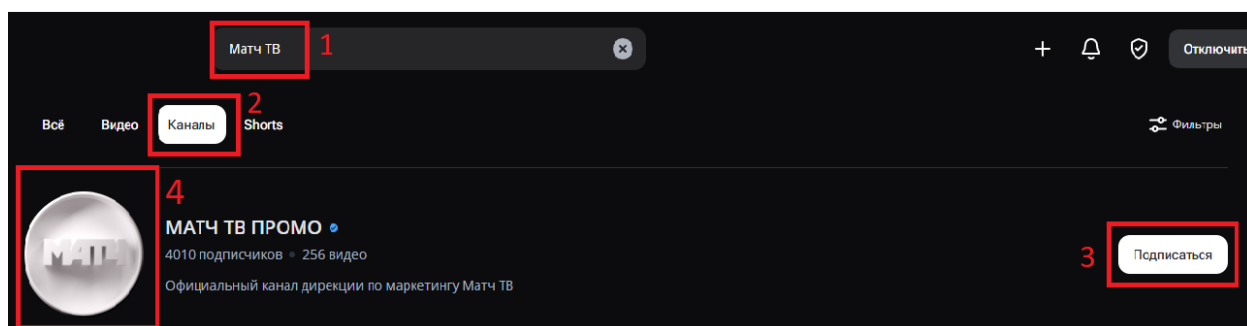


Рисунок 7 – Ввод поискового запроса «Матч ТВ», переключение на вкладку «Каналы», клик по кнопке «Подписаться» и переход в профиль канала



Рисунок 8 – Проверка статуса кнопки в профиле канала

2.1.3 Проверка взаимодействия с видео

Это самая большая часть тестов, в которой проверяется всё, что связано с просмотром. Сюда входит проверка самого плеера (пауза, смена качества,

полноэкранный режим), а также возможность ставить лайки и дизлайки, сохранять ролики в плейлист «Смотреть позже», копировать ссылки и отправлять жалобы на видео.

Тест 1. Поиск видео: Тест ищет ролики по слову «музыка», открывает первое видео и нажимает на кнопку паузы в открывшемся плеере (рис.9 и рис.10). Проверяется, что воспроизведение успешно приостановилось.

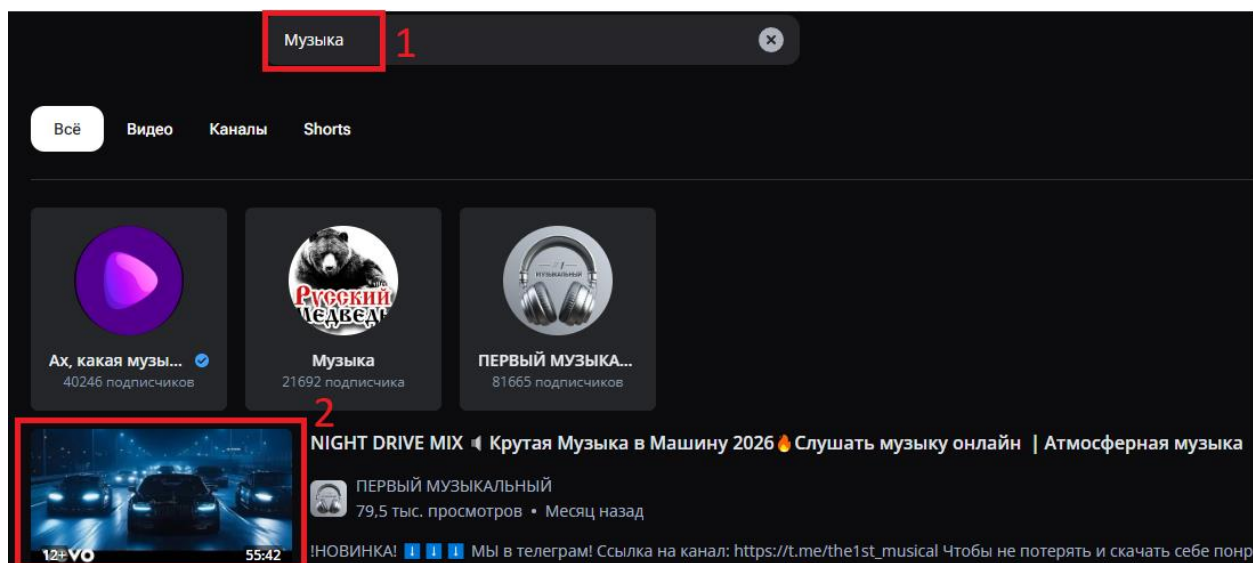


Рисунок 9 – Ввод поискового запроса «Музыка» и открытие первого видео

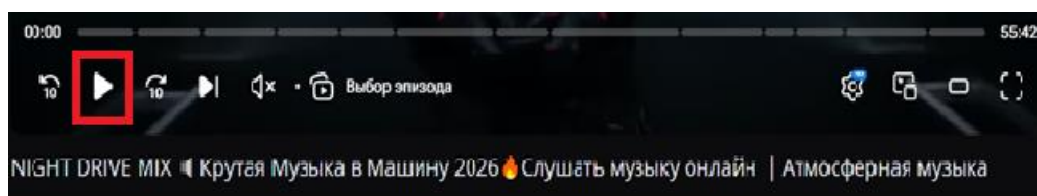


Рисунок 10 – Наведение на плеер и нажатие кнопки паузы

Тест 4. Отправка жалобы на видео: Тест ищет «фильмы», открывает первое видео и нажимает кнопку «Пожаловаться» (рис. 11 и рис. 12). Проверяется, что форма жалобы успешно открылась на экране, после чего тест закрывает эту форму (рис. 13).

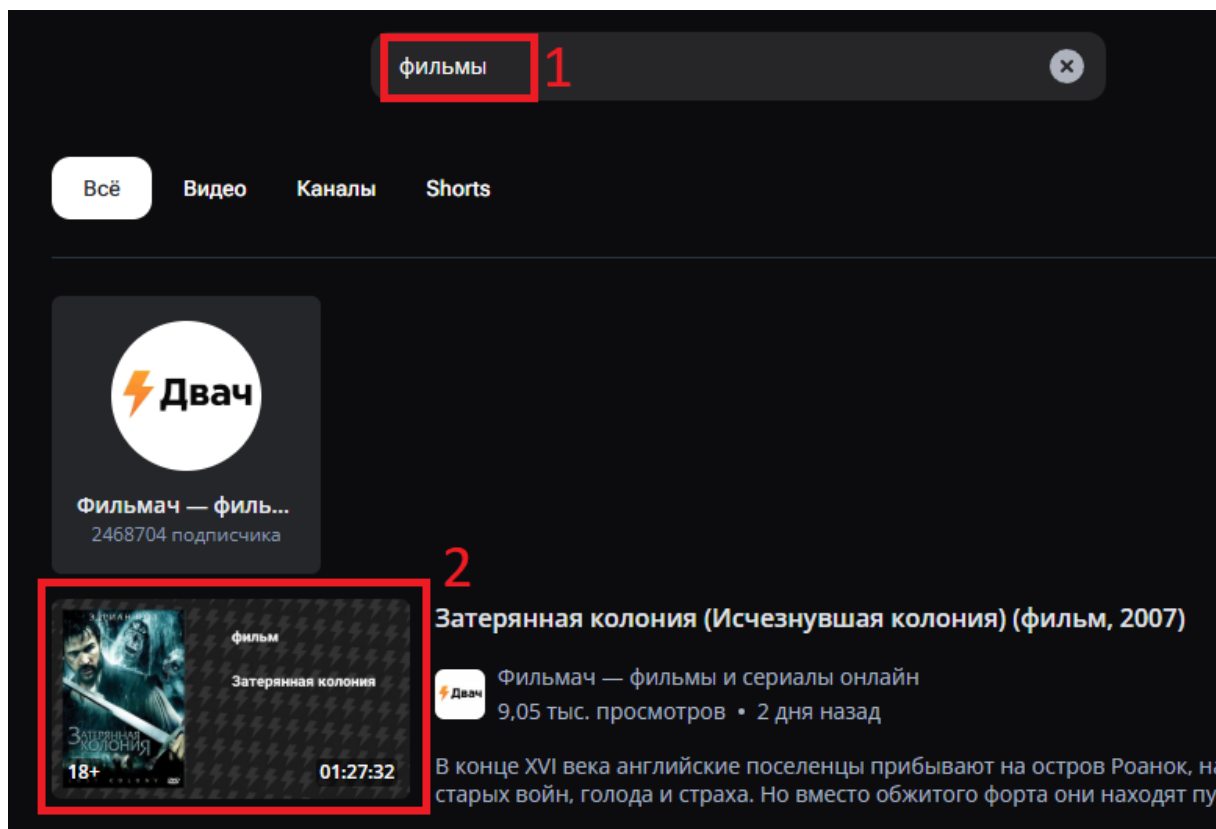


Рисунок 11 – Ввод поискового запроса «Фильмы» и открытие первого видео

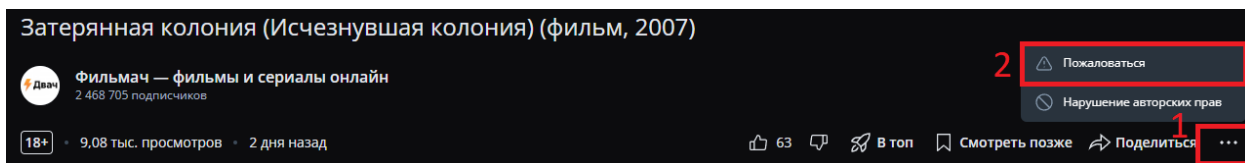


Рисунок 12 – Открытие контекстного меню и клик по кнопке «Пожаловаться»

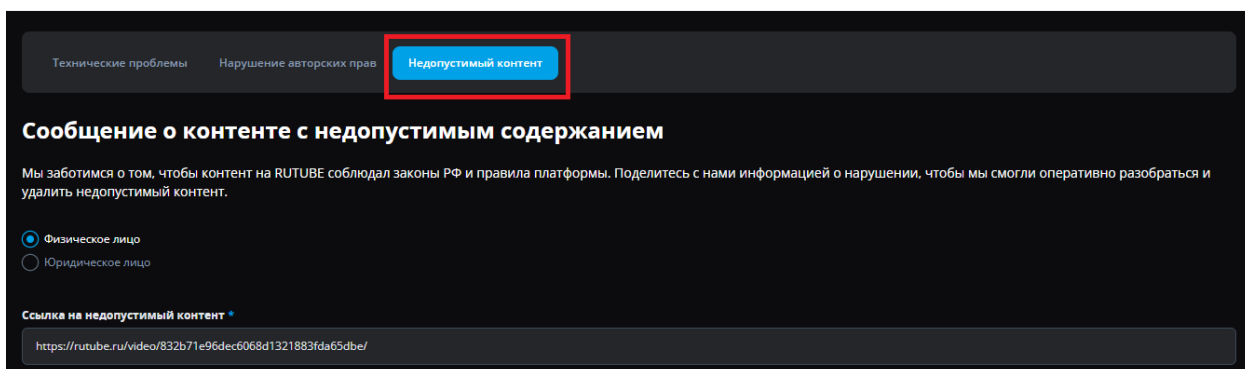


Рисунок 13 – Открытие формы жалобы на видео

Тест 7. Копирование ссылки на видео: Тест ищет «новости» и открывает первое видео (рис. 14). Затем нажимает кнопку «Поделиться» и выбирает

«Копировать ссылку» (рис. 15). После этого программа проверяет, что ссылка успешно сохранилась в буфер обмена компьютера.

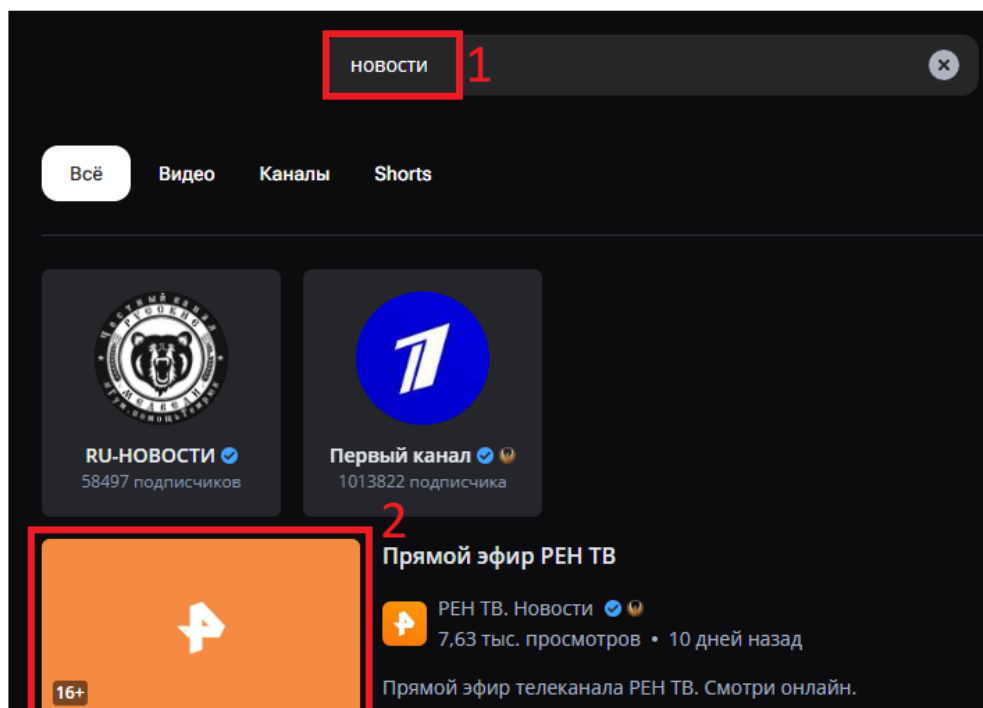


Рисунок 14 – Ввод поискового запроса «Новости» и открытие первого видео

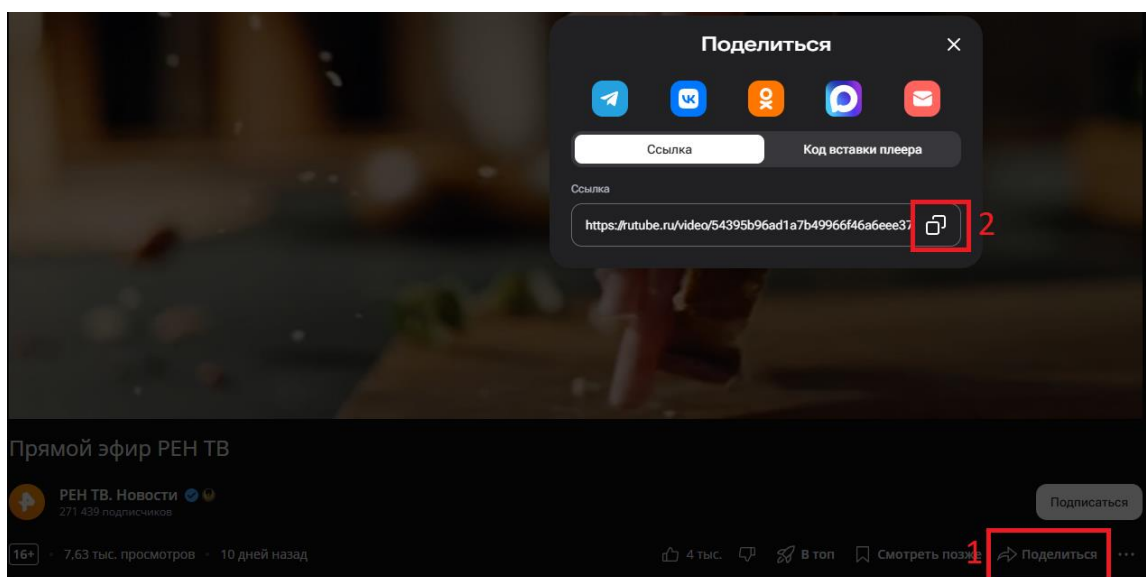


Рисунок 15 – Клик по кнопке «Поделиться» и «Копировать ссылку»

Тест 8. Добавление видео в «Смотреть позже»: Тест находит видео по запросу «музыка», открывает его контекстное меню и нажимает «Смотреть позже» (рис. 16). После этого переходит в раздел «Смотреть позже» в левом меню и проверяет, что ролик появился в плейлисте (рис. 17).

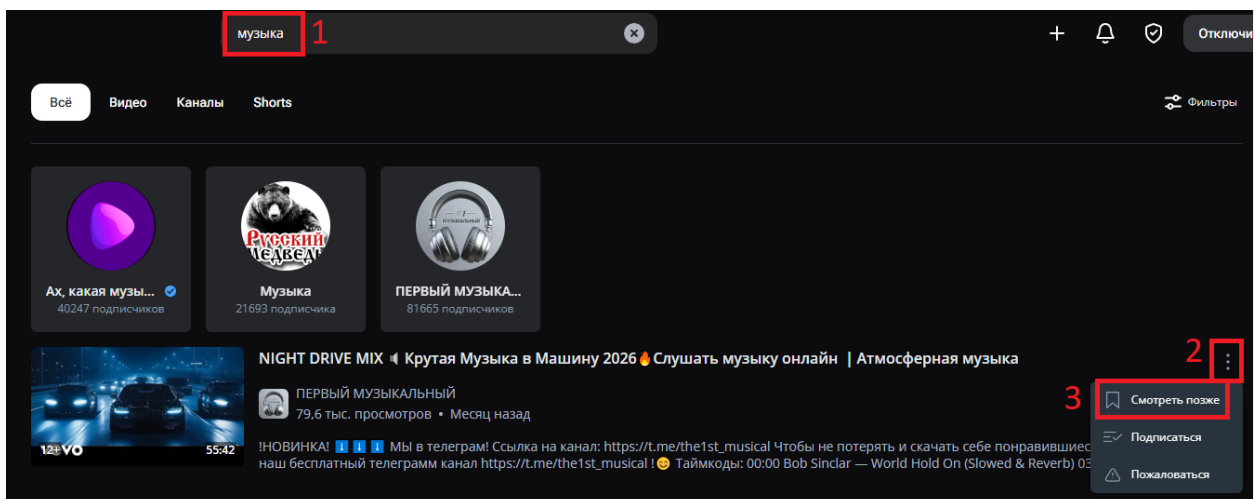


Рисунок 16 – Ввод поискового запроса «музыка», открытие меню и клик по кнопке «Смотреть позже»

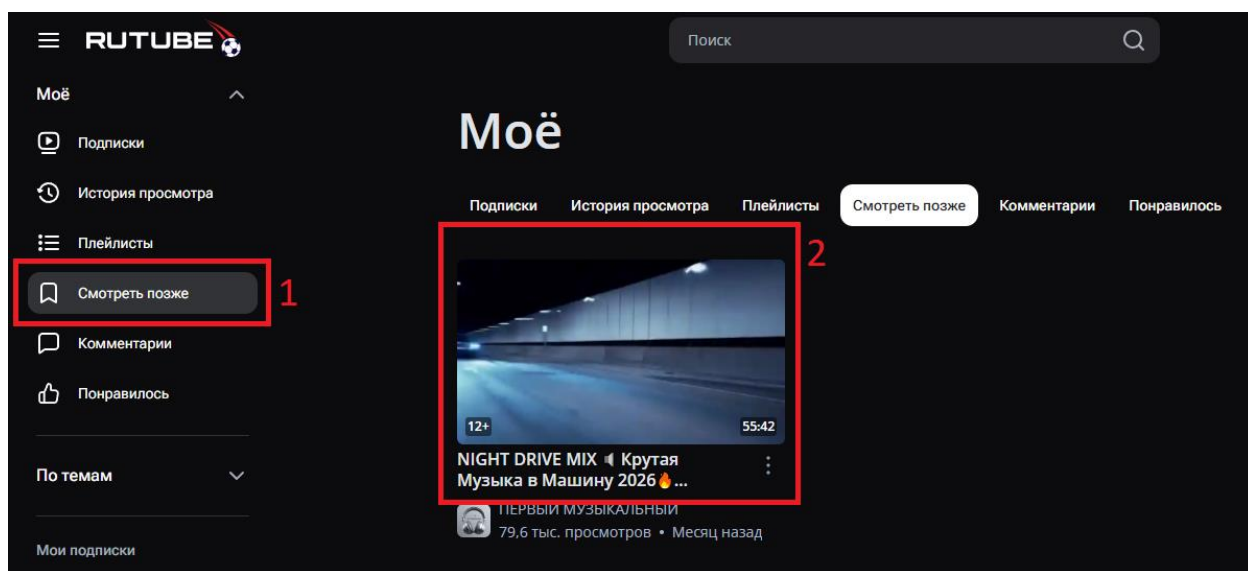


Рисунок 17 – Переход в раздел «Смотреть позже» и проверка появления ролика

Тест 9. Оценка видео (Лайк/Дизлайк): Тест открывает видео по запросу «музыка», нажимает кнопку «Лайк», а потом сразу же «Дизлайк» (рис. 18). Затем переходит в раздел «Понравилось» и проверяет, что этого видео там нет, так как система корректно учла последнюю оценку (рис. 19).

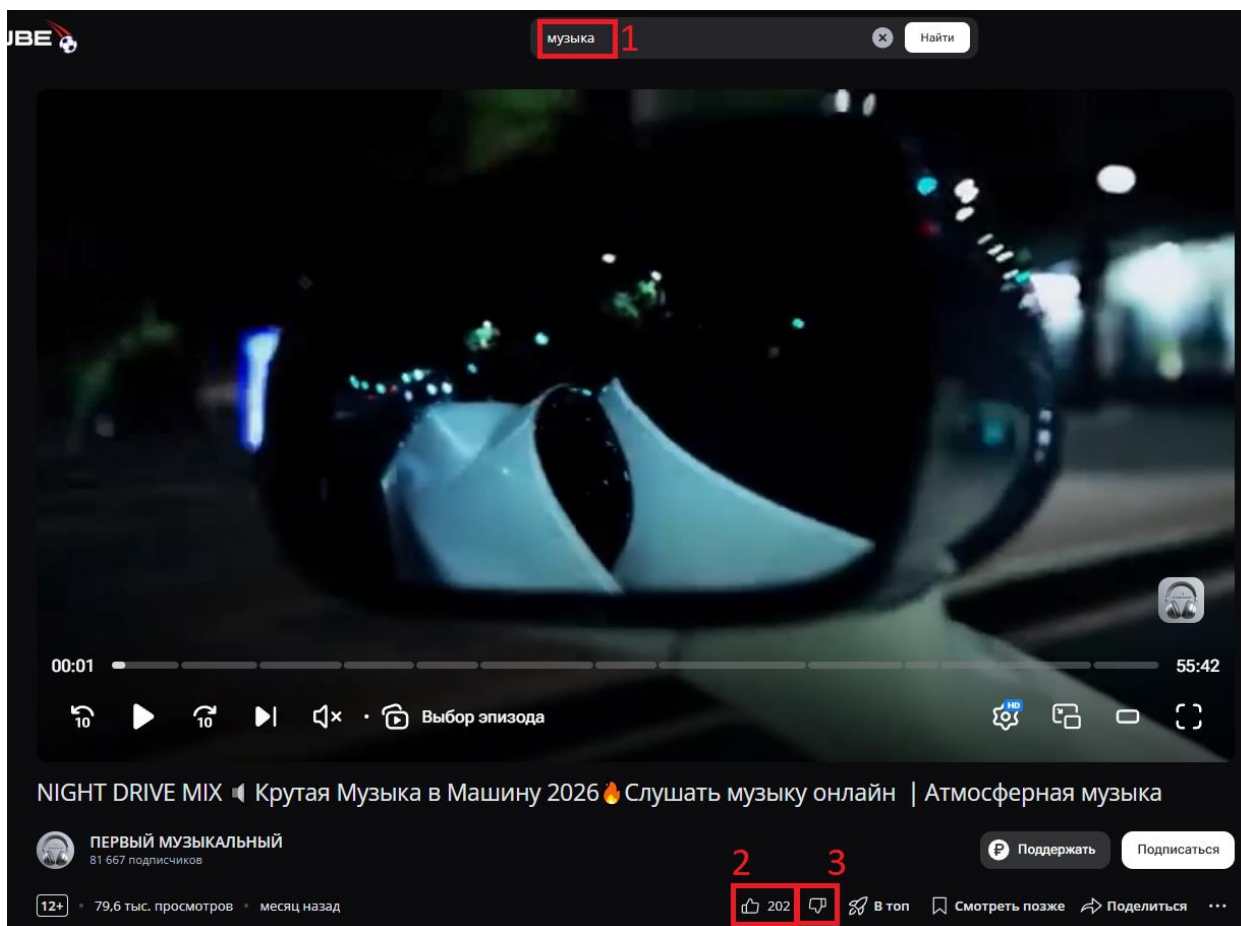


Рисунок 18 – Ввод поискового запроса «музыка», открытие видео, нажатие кнопки «Лайк» и «Дизлайк»

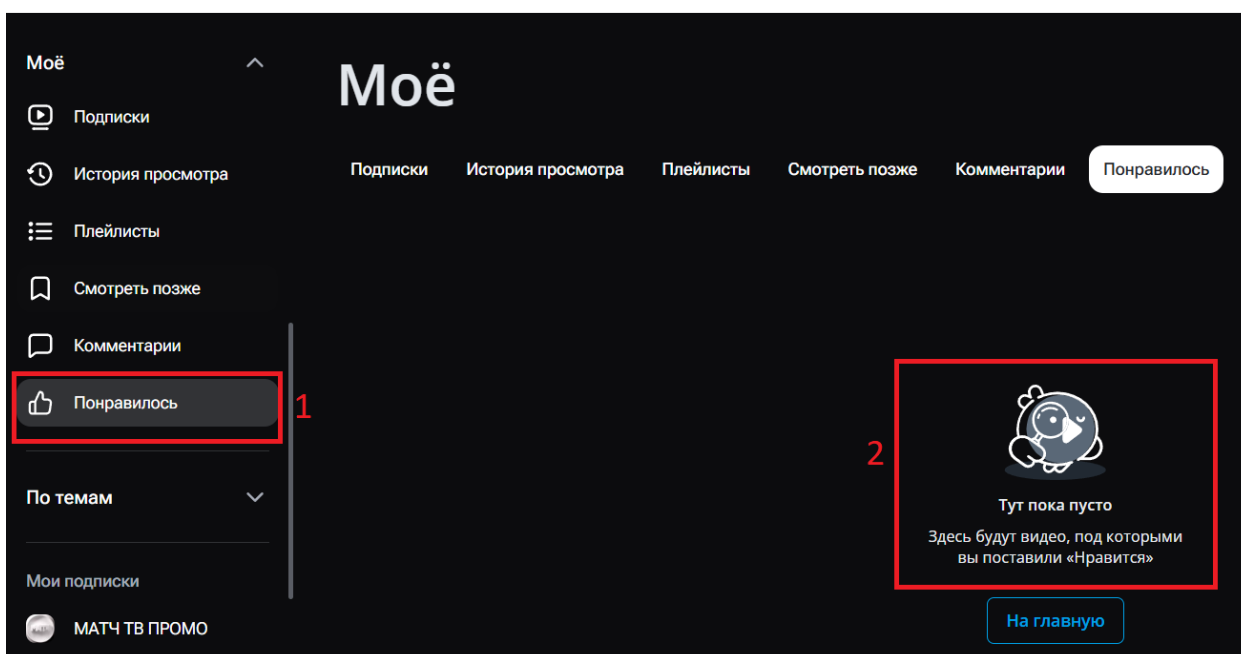


Рисунок 19 – Открытие раздела «Понравилось» и проверка отсутствия видео

Тест 10. Настройка видео: Тест открывает видео по запросу «музыка» и заходит в настройки плеера (шестеренка). Затем выбирает качество «480p» и нажимает кнопку «Полноэкранный режим» (рис. 20). В конце проверяется, что видео расширилось, а качество 480p успешно применилось.

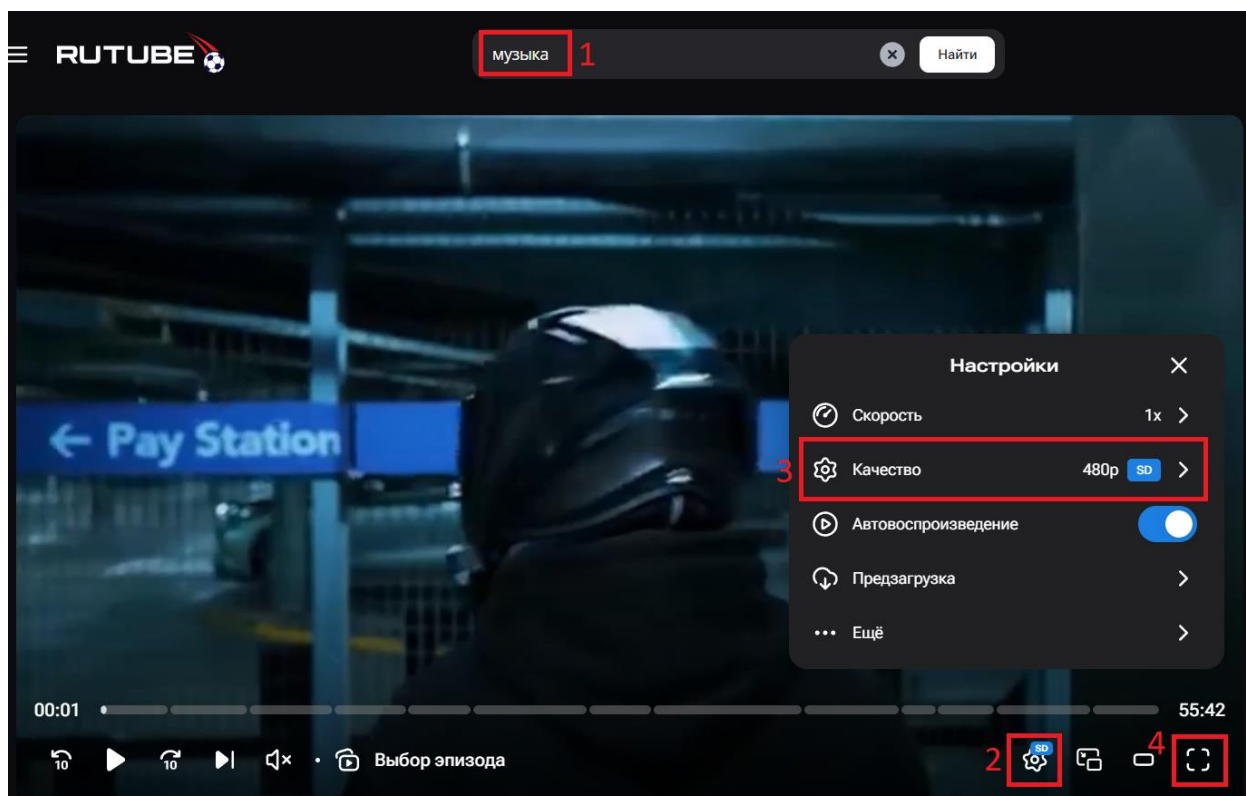


Рисунок 20 – Ввод поискового запроса «музыка», открытие видео, выбор качества «480p» и переход в «Полноэкранный режим»

2.1.4 Результат выполнения всех тестов

Все тесты были запущены вместе в среде разработки IntelliJ IDEA через JUnit 5. Для выполнения тестов, требующих авторизации профиля (подписки, лайки, жалобы, плейлисты), система перед каждым тестом автоматически загружала данные из файла cookies.json. Это позволило полностью пройти весь цикл проверок без ручного входа и ввода.

По итогу все тесты успешно прошли проверку и получили статус «Passed». Программа не выявила зависаний элементов интерфейса или ошибок при загрузке страниц, что говорит о стабильной работе проверенных функций сайта и правильной настройке самого кода автотестов.

2.2 Описание классов и методов

Архитектура проекта построена на основе Page Object Model (POM). Для удобства и читаемости весь программный код разделен на функциональные блоки: базовые классы, элементы интерфейса, страницы, утилиты и классы с тестами.

2.2.1. Блок базовых классов (base)

В этом разделе хранятся родительские классы, которые задают общую логику для всего проекта.

BaseTest — фундамент для всех тестовых классов. Содержит методы, которые выполняются до и после каждого теста:

- setUp(TestInfo testInfo) — начальная настройка: задает размер окна, открывает стартовую страницу, закрывает всплывающие окна, подгружает куки для авторизации и делает перезагрузку страницы (refresh) для применения настроек. Также автоматически фиксирует название запущенного теста в консоли через логгер.
- closePopups() — автоматический поиск и закрытие рекламных баннеров и окон с соглашением об использовании файлов куки.
- loadCookiesFromFile(String filePath) — программная авторизация тестового профиля путем подгрузки сохраненной сессии из JSON-файла.
- tearDown(TestInfo testInfo) — корректное завершение работы веб-драйвера и вывод в лог информации об окончании теста.

BasePage — родительский класс для всех страниц. Содержит метод:

- `isDisplayed()` — проверяет, успешно ли загрузилась требуемая страница на экране пользователя (ищет корневой элемент страницы).

`BaseElement` — родительский класс для интерактивных элементов. Содержит общие методы:

- `isDisplayed()` — проверяет, виден ли конкретный элемент на экране в данный момент, ожидая его появления до 10 секунд.
- `waitForLoad()` — заставляет тест сделать паузу и гарантированно дожждаться, пока элемент загрузится на странице.
- `getText()` — считывает и возвращает текстовое содержимое, которое написано внутри элемента.
- `getBaseElement()` — технический метод, дающий прямой доступ к элементу для выполнения нестандартных команд библиотеки Selenide.

2.2.2 Блок элементов интерфейса (*elements*)

В этом разделе находятся классы-обертки для стандартных веб-элементов сайта. Они наследуют общую логику от `BaseElement` и реализуют функции конкретных элементов веб-страницы.

`Button`, `Link`, `Tab` — классы для взаимодействия с кнопками, ссылками и вкладками. Содержат методы:

- `click()` — имитирует стандартное нажатие по выбранному элементу.
- `byText()`, `byClass()`, `byXpath()`, `byHref()` — статические методы поиска, которые помогают быстро найти нужный элемент на странице по его тексту, классу или адресу ссылки, избавляя от необходимости прописывать длинные пути.

Input – класс для работы с текстовыми полями ввода. Содержит методы:

- `fill(String value)` — умный ввод текста: метод сначала очищает поле (если есть кнопка очистки), затем печатает нужный текст и нажимает клавишу Enter.
- `clear()` — выполняет стандартную очистку поля от напечатанных символов.
- `withClearButton(Button button)` — связывает поле ввода с кнопкой-крестиком, чтобы можно было быстро сбрасывать старый текст в поиске.
- `byId(String id)`, `byName(String name)`, `byPlaceholder(String placeholder)`, `byClass(String className)` — методы для поиска нужного поля ввода по его параметрам в HTML-коде: по идентификатору, имени, тексту-подсказке или классу.

Dropdown – класс для взаимодействия с выпадающими списками.

Содержит методы:

- `selectOption(String optionText)` — кликает по выпадающему списку для его раскрытия, находит в нем вариант с нужным текстом и выбирает его.
- `byClass(String className)` — метод для поиска нужного выпадающего списка на странице по имени его класса.

VideoPlayer – класс для управления встроенным медиаплеером сайта.

Содержит методы:

- `clickWithJS(SelenideElement element)` и `showElement(SelenideElement element)` — скрытые технические функции, использующие JavaScript для взаимодействия с кнопками плеера. Это необходимо для обхода проблемы, когда элементы перекрыты другими объектами на странице.

- `getPlayer()` — возвращает готовый объект плеера для начала работы с ним.
- `pause()` — наводит мышь на плеер и ставит видео на паузу, если оно воспроизводится.
- `isPaused()` — проверяет, находится ли видео на паузе в данный момент.
- `openSettingsMenu()` — технический метод, который открывает панель настроек внутри плеера.
- `setQuality(String quality)` — открывает меню настроек плеера и выбирает нужное разрешение видео.
- `getCurrentQuality()` — считывает информацию из настроек плеера и возвращает текст с текущим качеством воспроизведения.
- `toggleFullscreen()` — наводит курсор на плеер и нажимает кнопку разворачивания видео на весь экран.
- `waitForLoad()` — заставляет тест дожидаться полной загрузки плеера, прежде чем выполнять с ним какие-либо действия.
- `hover()` — имитирует движение мыши по плееру, чтобы на экране появились скрытые элементы управления (кнопка паузы, шестеренка настроек и т.д.).

2.2.3 Блок страниц (pages)

В этом разделе находятся классы, которые описывают конкретные веб-страницы сайта. Каждый класс содержит методы для выполнения действий, доступных пользователю на соответствующей странице.

`VideoPage` — класс для работы со страницей просмотра видеоролика. Содержит методы:

- `switchToNewWindow()` — метод, переключающий внимание автотеста на новую открытую вкладку браузера.
- `getVideoPlayer()` — предоставляет доступ к объекту плеера для прямого управления воспроизведением.

- `getCurrentQuality()`, `isPaused()`, `setQuality(String quality)`, `pause()`, `toggleFullscreen()` — методы управления плеером, которые передают команды напрямую в плеер, не перегружая код самой страницы.
- `openMenu()` — нажимает на кнопку с тремя точками под видео для открытия контекстного меню.
- `addToWatchLater()` — открывает меню и нажимает на кнопку добавления видео в список «Смотреть позже».
- `copyLink()` — нажимает кнопку «Поделиться», а затем выбирает опцию копирования ссылки на ролик.
- `isLinkCopied()` — проверяет, успешно ли сохранилась ссылка на сайт `rutube.ru` в системном буфере обмена устройства.
- `like()`, `dislike()` — нажимают соответствующие кнопки оценки видеоролика.
- `reportVideo()` — открывает меню, нажимает кнопку «Пожаловаться» и переключается на открывшуюся вкладку с формой.
- `isComplaintFormDisplayed()` — проверяет, успешно ли загрузилась страница с формой для отправки жалобы на контент.
- `closeComplaintForm()` — закрывает текущую вкладку с формой жалобы и возвращает управление на основную вкладку с видео.
- `getVideoTitle()` — считывает и возвращает текстовый заголовок текущего видеоролика.
- `isFullscreen()` — проверяет по атрибутам кнопки разворачивания, открыто ли видео на весь экран в данный момент.

`ChannelPage` — класс для работы со страницей канала пользователя.

Содержит методы:

- `subscribe()` — проверяет наличие кнопки подписки, и если еще нет подписки, нажимает на нее.

- `isSubscribed()` — проверяет, изменился ли текст на кнопке подписки на статус «Вы подписаны».

`MainPage` — класс, представляющий стартовую страницу сайта. Содержит методы:

- `search(String query)` — вводит переданный текст в поисковую строку (или оставляет ее пустой, если текста нет), нажимает клавишу поиска и переходит на страницу результатов.
- `goToTop()` — открывает раздел популярных видеороликов «В топе» по прямой ссылке.
- `openMainPage()` — возвращает пользователя на главную страницу кликом по логотипу или прямым переходом по ссылке.

`PlaylistPage` — класс для взаимодействия со списками воспроизведения (плейлистами). Содержит методы:

- `likedPlaylist()`, `watchLaterPlaylist()` — статические методы, которые открывают страницы плейлистов «Понравилось» или «Смотреть позже» соответственно.
- `isVideoInPlaylist(String videoTitle)` — ищет на странице плейлиста карточку видеоролика с заданным названием и возвращает успешный результат, если она найдена.
- `isVideoNotInPlaylist(String videoTitle)` — убеждается, что видеоролика с таким названием нет в текущем списке.

`SearchPage` — класс для работы со страницей результатов поиска. Содержит методы:

- `openFilters()` — дожидается загрузки результатов, проверяет, скрыта ли панель фильтров, и если скрыта — нажимает кнопку «Фильтры» для ее отображения.

- `applyFilter(String filterValue)` — находит на панели фильтр с нужным названием, нажимает на него и дожидается его активации.
- `openFirstVideo()` — кликает по первому видеоролику в списке результатов для перехода к его просмотру.
- `getFirstVideoTitle(), getFirstVideoPublishDate(), getFirstVideoDuration()` — считывают с карточки первого видеоролика его название, дату публикации и длительность соответственно.
- `getSearchInputValue()` — возвращает текст, который в данный момент введен в поисковую строку.
- `goToChannelsTab()` — переключает результаты поиска на вкладку с найденными каналами.
- `subscribeToFirstChannelAndGoToChannel()` — нажимает кнопку подписки на первом канале в списке, а затем переходит на его страницу.
- `clearSearch()` — нажимает на крестик в строке поиска для удаления введенного текста.
- `searchAgain(String query)` — очищает строку поиска, вводит новый запрос и запускает его.
- `isFiltersOpened(), waitForResultsLoad(), findFilterOption(), waitForFilterActivation(), getFirstVideoAttribute()` — скрытые технические методы для проверки состояния элементов и поиска информации внутри карточек видео.

2.2.4 Блок тестов (*tests*)

В этом разделе хранятся сами сценарии автоматизированных проверок, разделенные по смыслу на несколько классов. Также внутри каждого теста есть пошаговое логирование действий пользователя

`SearchTests` – набор проверок поисковой системы. Содержит сценарии:

- `applyFilters()` — ищет новости, применяет фильтры по дате и длительности, затем проверяет, что первое видео в списке действительно соответствует этим строгим критериям.
- `changeSearchQuery()` — делает запрос, стирает его, вводит новый и убеждается, что текст в строке и результаты выдачи успешно обновились.
- `emptySearch()` — пытается выполнить поиск без ввода текста, переходит в другой раздел и возвращается обратно, чтобы проверить, что сайт не выдает ошибку и продолжает работать корректно.

`SubscriptionTests` – набор проверок для работы с подписками. Содержит сценарии:

- `searchChannelAndSubscribe()` — находит через поиск заданный канал, оформляет на него подписку из списка результатов и проверяет на странице канала, что подписка оформлена.

`VideoTests` Набор – проверок для работы с плеером и оценки видеороликов. Содержит сценарии:

- `searchVideoAndPause()` — находит видео, включает его и ставит на паузу, проверяя, что воспроизведение действительно остановилось.
- `sendComplaint()` — открывает меню видеоролика, вызывает форму отправки жалобы на контент и закрывает ее без ошибок.
- `copyVideoLink()` — нажимает кнопку «Поделиться» и проверяет, что ссылка скопировалась в системный буфер обмена.
- `addToWatchLater()` — сохраняет ролик в плейлист для отложенного просмотра и проверяет его фактическое появление в этом плейлисте.
- `likeAndDislike()` — ставит ролику лайк, затем меняет оценку на дизлайк и убеждается, что видео пропало из списка понравившихся.

- `videoSettings()` — запускает ролик, выставляет конкретное качество изображения и переводит плеер в полноэкранный режим работы.

2.2.5 Блок утилит (*utils*)

В этом разделе находятся вспомогательные классы с инструментами общего назначения.

`Utils` – класс со статическими методами. Содержит методы:

- `parseDuration(String duration)` — принимает текстовую строку с длительностью видео (например, в формате минут и секунд) и математически переводит ее в общее количество секунд для удобства проведения проверок в тестах.

`Logger` — класс для ведения журнала событий (логирования) в консоли.

Содержит методы:

- `info(String message)` — выводит в консоль текстовое сообщение о том, какое действие сейчас выполняет автотест.
- `testStart(String testName)` — визуально отделяет начало нового теста в консоли с помощью разделительной линии и выводит его название.
- `testEnd(String testName)` — фиксирует завершение теста и также выводит разделительную линию для удобства чтения логов.

2.2.6 Диаграмма классов (*UML*)

Для наглядного представления структуры разработанного проекта автоматизации тестирования и демонстрации связей между его компонентами была построена общая UML-диаграмма, а также классов элементов и страни.

Данная схема отражает иерархию наследования, а также взаимодействие базовых классов, элементов интерфейса, страниц веб-ресурса, тестовых сценариев и утилит. Использование объектно-ориентированного подхода и шаблона Page Object Model (POM) позволило изолировать логику работы с веб-элементами от самих автотестов.

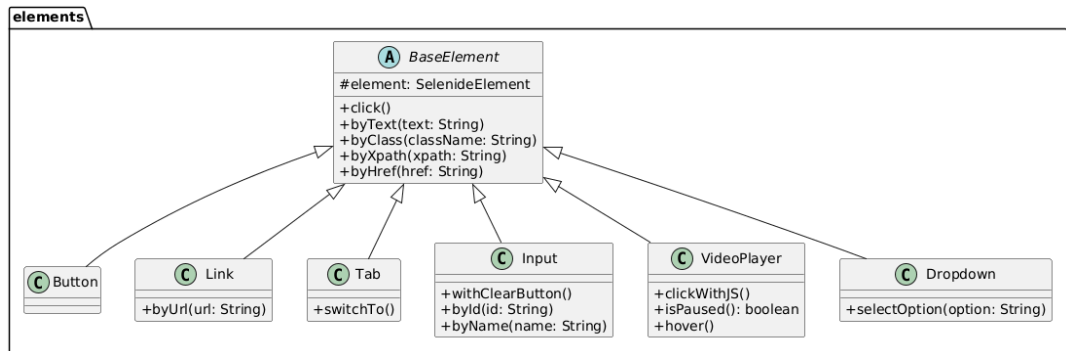


Рисунок 21 - Диаграмма классов элементов (Пакет elements)

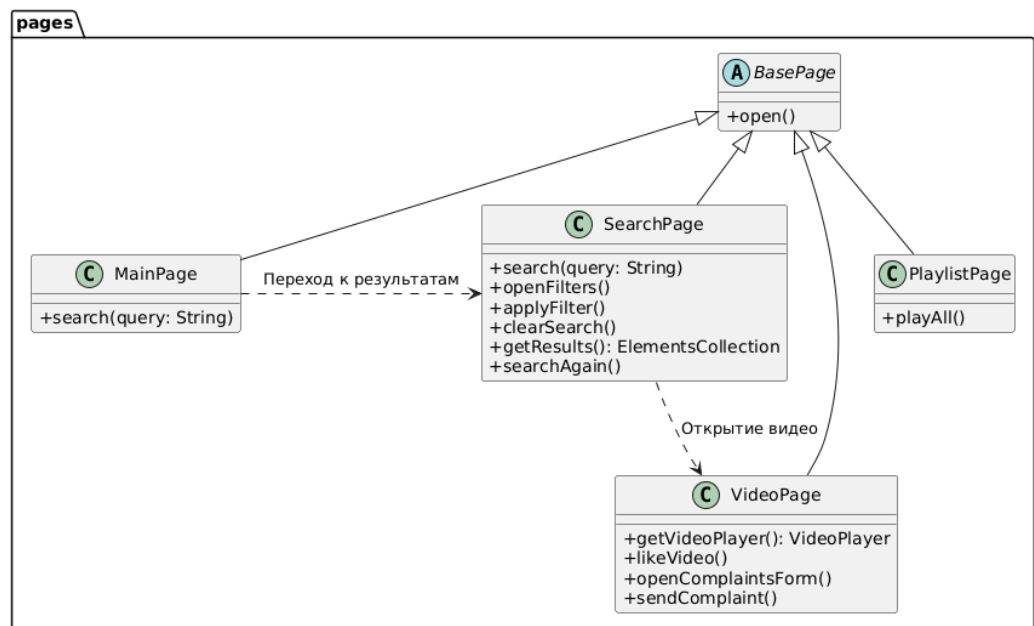


Рисунок 22 - Диаграмма классов страниц (Пакет pages)

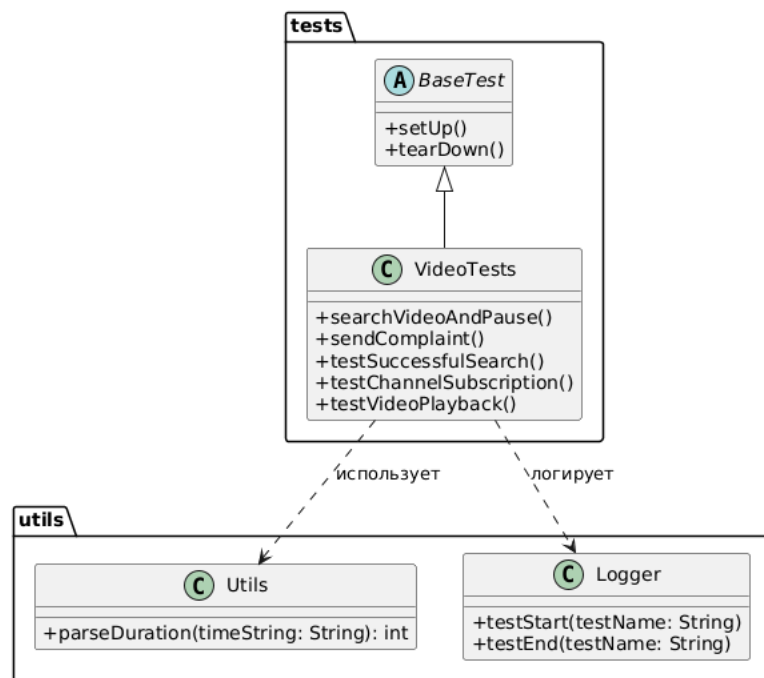


Рисунок 23 - Диаграмма классов тестов и утилит (tests и utils)

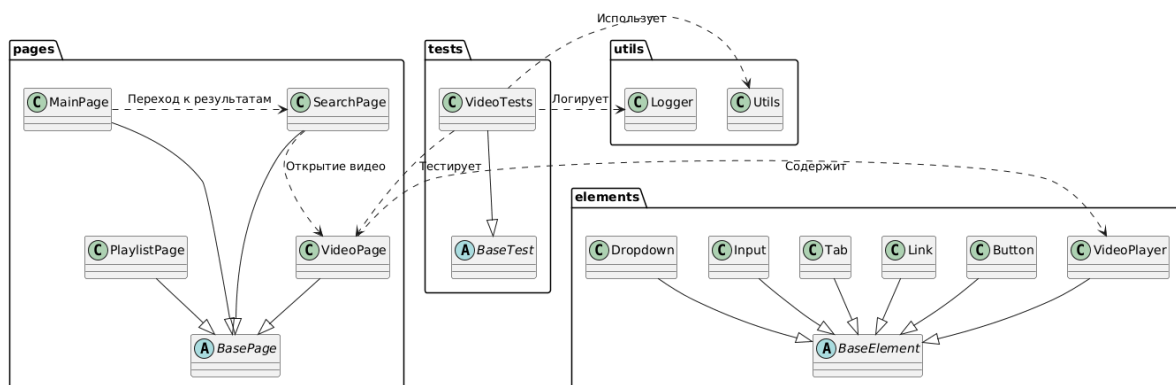


Рисунок 24 - Общая UML-диаграмма

2.3 Индивидуальная работа

В ходе практики я занималась проектированием, разработкой и стабилизацией автоматизированных тестов. Ниже приведено подробное описание реализованного функционала.

2.3.1 Составление чек-листа для проверок

Для систематизации процесса тестирования я сделала чек-лист. В нем собраны основные сценарии, шаги для их выполнения и ожидаемые результаты. Это помогло ничего не упустить при написании кода.

2.3.2 Настройка автоматической авторизации

Для обхода ручного ввода данных для входа в аккаунт при каждом запуске тестов, я реализовала механизм автоматической авторизации. В класс BaseTest был добавлен метод loadCookiesFromFile, который использует библиотеку ObjectMapper для чтения сохраненных сессионных кук из файла cookies.json.

Теперь тесты, требующие авторизации, запускаются намного быстрее и работают стабильнее.

2.3.3 Рефакторинг по требованиям преподавателя

Чтобы сделать код чище и привести его к паттерну Page Object, я выполнила ряд изменений:

- Из BaseElement удалила метод click(), оставив его только в конкретных классах элементов, где он нужен (Button, Link и Tab). В классе Input метод pressEnter() перенесла внутрь fill().
- Все XPath-локаторы вынесла из методов в константы private static final String внутри соответствующих классов (например, в MainPage, SearchPage, VideoPlayer).

- В классе `VideoPlayer` конструктор был изменен на `private`, чтобы исключить прямое создание экземпляров извне.
- В классе `SearchPage` метод `applyFilter()` был декомпозирован: ожидание применения фильтра и другие действия вынесены в отдельные методы.
- Логика метода `searchAgain()` была упрощена, и теперь все действия выполняются внутри единого вызова.
- Вместо множества отдельных методов для получения данных о первом видеоролике (название, дата публикации) был создан один общий `private` метод. Он принимает аргумент для извлечения конкретного атрибута, а в тестах используются отдельные геттеры, обращающиеся к этому приватному методу.
- Общий класс `RutubeTests` был разбит на изолированные модули в зависимости от области тестирования: `SearchTests`, `VideoTests` и `SubscriptionTests`.

2.3.4 Исправление и стабилизация тестов

Были внесены правки во 2 и 3 тесты, а также исправлена логика работы тестов 7, 8, 9 и 10:

- Добавила закрытия всплывающих окон (попапов и уведомлений о куках) через метод `closePopups()` в `setUp()` при каждом тесте.
- В тесте на копирование ссылки (тест 7) проверка UI-уведомления была заменена на прямое чтение содержимого буфера обмена через `Selenide.clipboard()`, что сделало проверку более надежной.
- В тесте 8 обновила логику взаимодействия со страницей плейлистов, исправив `XPath`-локаторы для более надежного поиска элементов.

- В тесте 9 взаимодействия с видеоплеером (оценка "Нравится"/"Не нравится") добавила явные паузы `Selenide.sleep()`, чтобы синхронизировать работу кода с интерфейсом Rutube.
- Обновление проверочных данных (изменение ожидаемого разрешения качества видео с 1080p на 720p), чтобы тесты успешно проходили, так как не у всех видео есть возможность выбрать разрешение 1080p.

3. ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В данной главе представлены основные технологий, языки программирования и программные инструменты, которые применялись для

реализации проекта автоматизированного UI-тестирования в рамках учебной практики:

1. Java: Основной язык программирования, на котором написан весь проект.
2. Selenide: Фреймворк для управления браузером. С его помощью реализован весь процесс UI-тестирования: поиск элементов на сайте Rutube, клики по кнопкам, ввод текста и ожидания загрузки страниц.
3. JUnit 5: Библиотека для написания и запуска тестов. Она использовалась для организации структуры тестов, управления порядком их выполнения (через аннотации) и проверки финальных результатов.
4. Maven: Инструмент для сборки проекта. Он автоматически скачивал и подключал все необходимые внешние библиотеки (Selenide, JUnit и др.) через конфигурационный файл `pom.xml`.
5. Jackson (класс `ObjectMapper`): Библиотека для работы с форматом JSON. Она понадобилась для настройки автоматической авторизации — с ее помощью программа считывала сессионные куки из файла `cookies.json`.
6. XPath: Язык запросов для поиска элементов на веб-странице. Использовался для написания локаторов кнопок, полей ввода и карточек видео в HTML-коде Rutube.
7. IntelliJ IDEA: Среда разработки, в которой писался и редактировался код, проводился рефакторинг архитектуры и запускались тесты.
8. Git и GitHub: Система контроля версий. Использовались участниками команды для совместной удаленной работы над проектом, ведения истории изменений (коммитов) и хранения актуальной версии автотестов в общем репозитории.
9. Google Chrome: Браузер, используемый для выполнения тестов.

4. ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на <ваша_оценка>.

ЗАКЛЮЧЕНИЕ

В ходе прохождения учебной практики была успешно достигнута главная цель работы — создан рабочий проект для автоматизированного UI-тестирования веб-сайта Rutube. В процессе работы над проектом были выполнены все задачи, поставленных в начале практики.

Во-первых, в рамках первой задачи были изучены основы автоматизации тестирования веб-приложений. В качестве основного языка программирования применялась Java, а для управления браузером и имитации действий реального пользователя была освоена библиотека Selenide. Для организации проверок и сборки проекта также были изучены и применены инструменты JUnit 5 и Maven.

Во-вторых, была успешно решена задача по проектированию правильной архитектуры проекта. Мы применили подход Page Object Model (POM), благодаря чему весь код был логично разделен по папкам: базовые настройки, элементы, страницы и сами тесты. Такое разделение позволило отделить логику кликов и поиска элементов от логики самих проверок, что сделало код более читаемым и аккуратным. В рамках этой же работы было выполнено программное описание основных страниц (главная, поиск, страница канала, страница видео, плейлисты) и всех нужных компонентов сайта, включая сложный класс для управления медиаплеером.

В-третьих, была реализована задача по настройке автоматической авторизации. Чтобы тесты не прерывались на этапе ручного входа в аккаунт, был написан метод для чтения сессионных кук из файла cookies.json. Это решение позволило тестам выполняться в полностью автоматическом режиме без участия человека.

В результате было написано 10 тестов, которые покрывают ключевые пользовательские сценарии на Rutube. Реализованные автотесты проверяют

корректную работу поисковой строки, применение фильтров, подписку на каналы, взаимодействие с видеоплеером (смена качества, полноэкранный режим, пауза), а также функционал оценок, добавления роликов в плейлисты и отправки жалоб. При запуске через среду разработки все тесты отработали корректно, интерфейс сайта не зависал, и все проверки получили статус успешного прохождения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Selenide [Электронный ресурс]. URL: <https://selenide.org> (дата обращения 10.07.2026)
2. Maven [Электронный ресурс]. URL: <https://maven.apache.org/index.html> (дата обращения 12.07.2026)
3. Page Object Model [Электронный ресурс]. URL: https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/ (дата обращения 11.07.2026)
4. Junit [Электронный ресурс]. URL: <https://junit.org> (дата обращения 11.07.2026)